

OVERWATCH

A self-running AI news crew for Instagram — 8 agents, one pipeline, \$0 budget

Scope: design and build plan for a fully automated news channel operated by a crew of 8 AI agents orchestrated in **n8n (self-hosted, Community Edition)**: Scraper, Scoring, Content-Writer, Fact-Checker, Grouping Tool, Media-Generating, Caption-Generator and Scheduling agents. Every headline is turned into a 3,000+ word article, fact-checked, grouped, rendered into a themed 10-card Instagram carousel, captioned and published at data-driven best times — with **zero recurring cost**.

100% Free Stack

n8n Self-Hosted

8 Agents

100+ Headlines / Cycle

30-40 Candidates per Cycle

3,000+ Words per Article

10-Card Carousels

Fact-Checked

Table of Contents

1. Executive Summary	3
2. The Zero-Budget Stack	5
2.1 Free components and their roles	5
2.2 What you will never pay for (and why)	6
3. System Architecture	7
3.1 Pipeline overview	7
3.2 Shared storage — the backbone	7
3.3 The OverWatch Watchtower	7
4. n8n Implementation Map	8
4.1 Workflow inventory	8
4.2 Credentials to configure once	9
4.3 Module template for every agent	9
5. Agent 1 — Scraper Agent	10
6. Agent 2 — Scoring Agent	12
7. Agent 3 — Content-Writing Agent	14
8. Agent 4 — Fact-Checker Agent	16
9. Agent 5 — Grouping Tool	17
10. Agent 6 — Media-Generating Agent	18
11. Agent 7 — Caption-Generator	20
12. Agent 8 — Scheduling Agent	21
13. Shared Storage & Data Model	23
14. Watchtower, Alerts & Monitoring	24
15. End-to-End Workflow & Queue Manager (41 steps)	25
16. Phase 2 Roadmap (still \$0)	27
17. Quality, Tests & Reliability (new in v1.1)	28
18. Risks & Mitigations	30
19. 30-Day Build & Test Plan	32
20. Channel KPIs & Targets	33
21. Legal & Compliance	34
Appendix A — n8n Node Cheat-Sheet	34
Appendix B — Prompt Library (copy-paste)	35
Appendix C — Kickoff Checklist	37

1. Executive Summary

What changed in v1.1 (this revision): (1) new **Queue Manager** — scoring now feeds a candidate pool, and a priority×freshness engine decides what actually publishes, so the 30–40-per-cycle output can never pile into a stale backlog; (2) **dense zoom typography** — the 3,000-word article fits 10 cards via ~340-word dense blocks users pinch-zoom (verified in a pre-launch phone test); (3) **Cloudflare R2 media hosting** — Instagram requires public HTTPS image URLs, which the n8n tunnel cannot provide reliably; (4) **LLM failover** (Gemini → Groq → local Ollama) with a Watchtower health flag; (5) **golden-set regression tests** for the rubric and fact-checker; (6) **human-review bridge** for test-phase approvals; (7) **Insights feedback loop** for daypart/caption tuning; (8) **security hardening** checklist; (9) **cross-cycle dedup** via embedding cache; (10) detailed change-log in §17.7.

OverWatch is an autonomous AI news channel that runs itself: it collects more than one hundred news headlines per cycle from free RSS feeds, scores them against a strict engagement-focused rubric, keeps only the 30–40 strongest, writes a 3,000+ word article for each, fact-checks every article against credible sources, groups headline+article into auditable units, renders each group into a themed 10-card Instagram carousel (including the full article), generates a platform-native caption, and publishes on a data-driven schedule.

Every component is free. n8n Community Edition is self-hosted on free infrastructure; all AI calls run on generous free tiers (Google Gemini API, Groq); images come from a no-key image API (Pollinations) with local Pillow compositing; publishing uses Meta’s free Instagram Graph API; alerts go through a free Telegram bot; the database is SQLite — one file, no server.

1.1 System at a glance

Dimension	Design decision
Orchestrator	n8n Community Edition, self-hosted (Docker) on Oracle Cloud Always Free ARM VM (4 OCPU / 24 GB RAM) — unlimited workflows and executions, free forever.
Ingestion	16 curated RSS feeds (Google News topic & search feeds, BBC, Guardian, DW, Al Jazeera, France 24, TechCrunch, The Verge, Ars Technica, HN, Reddit) → 100–160 raw headlines per cycle.
Selection	Technical dedup (hash + semantic) then a 7-KPI rubric scored by an LLM; weighted 0–100; keep score ≥ 72, max 8 per topic → 30–40 candidates enter a pool managed by the Queue Manager (§15).
Articles	3,200–3,500 words (minimum 3,000 enforced by a hard gate), 8-section anatomy engineered for engagement.
Verification	Claim-by-claim fact-check against the source article + independent search results; ≤ 2 rewrites per article, then quarantine.
Packaging	Each group (headline + article + audit trail) becomes a 10-card 1080×1350 (4:5) carousel under one strict visual theme.
Publishing	Instagram Graph API, carousel containers, publish_time scheduling; max 3 posts/day, spaced ≥ 3 h, at best-time dayparts; 25-post/24 h platform cap respected.
Ops	Watchtower workflow: health sweep every 15 min, Telegram alerts, daily digest, kill-switch flags, SQLite audit log of every decision.
Cost	US\$0.00 recurring. Only non-cash inputs: a few hours of setup, your time running the 30-day test plan, and an Instagram Business account.

1.2 The eight agents and their hand-offs

Agent	Input → Output	Contract with the next stage
1 Scraper	RSS feeds → normalized headline records	≥ 100 unique, de-duplicated headline records in <code>headlines</code> , each with source URL and topic.
2 Scoring	Raw headlines → ranked shortlist	30–40 scored records (0–100 + subscores + reason) enter the candidate pool ; the rest are discarded to <code>audit_log</code> with reasons.

3 Content-Writer	Shortlisted headline → draft article	≥ 3,000-word markdown article, 8 sections, engaging hooks, source-linked, per headline.
4 Fact-Checker	Draft article → verified article	Verdict (pass / rewrite / quarantine) + list of verified claims and citations; rewrites capped at 2.
5 Grouping	Verified article → group unit	One <code>groups</code> row with audit ID <code>0W-yyyymmdd-NN</code> and a saved audit bundle (JSON + PDF).
6 Media-Gen	Group → 10-card carousel deck	10 PNG cards 1080×1350 (headline + full article), strict theme, stored per group.
7 Caption	Deck + article → caption	≤ 2,200-character caption with hook, summary, CTA, and hashtag block in <code>captions</code> .
8 Scheduler	Complete post → live Instagram post	Best-time slot assigned, Graph API containers created & published, permalink + status in <code>posts</code> .

Zero-budget rule applied strictly: every tool in this plan has a genuinely free tier that is sufficient for the workload defined here. Nothing in Section 2 requires a card, a trial, or a “free month that expires.” The only irreversible change is converting your Instagram account to Business/Creator mode and linking it to a Facebook Page — both free and required by Meta’s API.

2. The Zero-Budget Stack

2.1 Free components and their roles

Component	Free tier	Role in OverWatch	Why it fits the 0-budget rule
n8n Community Edition	Free, self-hosted (Docker)	Orchestrates all 8 agents; ~12 workflows, unlimited executions.	Fair-code license allows free commercial use for your own business; only reselling n8n hosting requires a license. No user caps, no execution limits when self-hosted.
Oracle Cloud Always Free	Free forever: 4 OCPU ARM / 24 GB RAM VM	Hosts n8n, SQLite, render scripts, n8n tunnel replacement.	One of the few genuinely permanent free VMs. Ampere A1 4-core/24 GB instance is free indefinitely (subject to availability; home PC is the fallback).
Cloudflare Tunnel	Free	Exposes n8n webhooks (for Telegram commands and Meta callbacks) without opening ports or buying a domain.	Zero-config HTTPS URL like <code>trycloudflare.com</code> or a free <code>*.trycloudflare.com</code> / your own free domain.
Google Gemini API	Free tier: ~250 req/day (API key) up to ~1,000–1,500 req/day (OAuth); 15–30 RPM	Primary LLM: scoring, writing, fact-check verdicts, captions (Gemini 2.5 Flash / Flash-Lite).	No card required, no expiry; quota math in \$7.6 fits a 48 h build cycle for 30–40 articles.
Groq (LPU inference)	Free: ~30 RPM, ~1,000–14,400 req/day depending on model	Secondary/fallback LLM for short outputs (scores, captions, verdicts) and overflow load.	No card; OpenAI-compatible so it drops into n8n HTTP nodes as a second provider.
Ollama (optional)	Free, runs on the 24 GB VM	Local fallback for embeddings (semantic dedup) and lightweight tasks if cloud quotas run out.	No API at all — runs on hardware you already have free.
Pollinations.ai	Free, no API key	Generates background art for the 10 cards (Flux models).	URL-based API (<code>image.pollinations.ai/prompt/...</code>); unlimited basic use; fallback: CSS gradient + Pillow pattern art.
Pillow (Python)	Free, open source	Renders 1080×1350 cards: layout, typography, safe zones, branding.	Local compositing guarantees deterministic, theme-consistent output — and zero per-card cost.
SQLite	Free	Shared storage — all agent hand-offs, queues, audit log.	One file, zero administration, perfect for a single-process n8n host.
Instagram Graph API	Free	Publishes carousel posts with <code>publish_time</code> scheduling.	Meta does not charge for API calls; you only need a Business/Creator account

			+ linked Facebook Page + free Meta Developer app.
Telegram Bot	Free	Alerts, daily digest, remote kill-switch commands.	BotFather gives you a token in 2 minutes; no hosting needed for the bot itself.
Canva (free) / Google Fonts	Free	Design once: brand kit, theme templates, fonts.	Design assets are made once, then reused forever by Pillow.
Meta Business Suite	Free	Fallback manual posting during Phase-0 testing, plus analytics cross-check.	Lets you keep publishing even if the API path is still pending app review.

2.2 What you will never pay for (and why)

- **No n8n Cloud subscription.** Self-hosting gives unlimited workflows/executions; the Community Edition license covers running your own channel commercially.
- **No LLM bill.** The whole cycle needs roughly 480 Gemini requests per 48 h build window (~240/day) — inside the free tier with OAuth (1,000/day) and manageable even on the 250/day key tier by splitting batches across days or using Groq for short tasks.
- **No image bill.** Pollinations is keyless; every card is a free AI background + deterministic Pillow overlay.
- **No hosting bill.** Oracle Always Free VM (or your own PC behind Cloudflare Tunnel).
- **No scheduling tool.** n8n's Schedule Trigger + Graph API `publish_time` replace Buffer/Later (\$15+/mo).
- **No analytics tool.** Graph API Insights + a free Telegram summary replace paid dashboards.

One honest caveat: Oracle Always Free ARM capacity is regional and occasionally unavailable; the plan ships with two fallback hosts (any old laptop/PC at home, or a Raspberry Pi 4/5 class device). All components are Docker-friendly and portable.

3. System Architecture

3.1 Pipeline overview

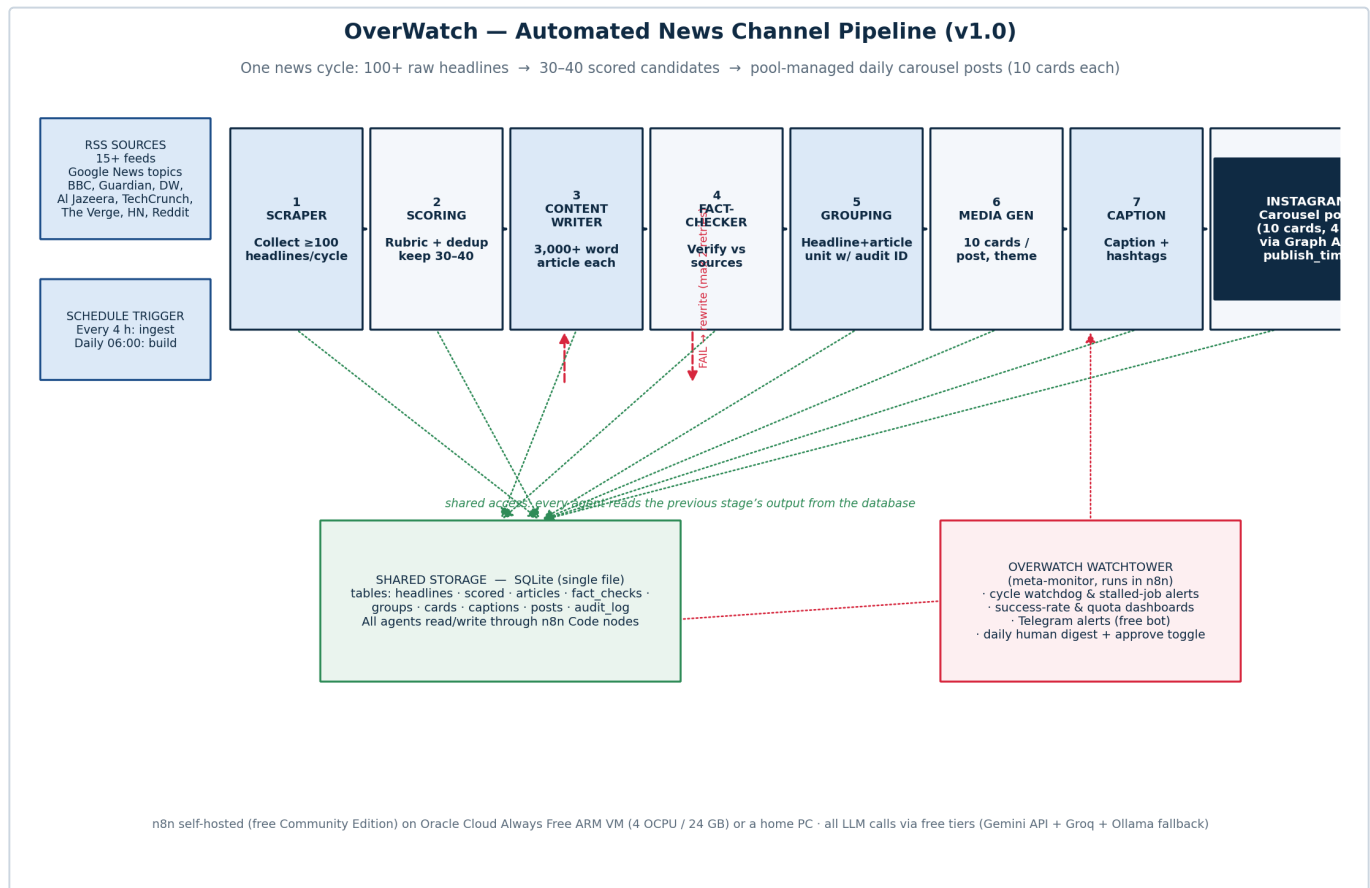


Figure 1 — OverWatch pipeline: 8 agents in a strict sequence, sharing one SQLite database, supervised by the Watchtower. The fact-checker can bounce an article back to the writer (max 2 retries).

3.2 Shared storage — the backbone

All agents communicate through **one SQLite database file** (`/data/overwatch.db`) rather than passing JSON between workflows. This gives every agent read access to every stage's output (the “shared storage with access to all agents” requirement), makes re-runs idempotent (status fields), and gives you a complete audit trail for free. n8n **Code nodes** (Python or JavaScript, built-in `sqlite3` / `better-sqlite3`) execute all SQL; no database service is needed. Schema in §13, Figure 4. In v1.1 the storage layer also hosts the **candidate pool** and **publish queue** (Tables 13.2–13.3) that decouple scoring volume from the 3-posts/day publishing rhythm.

3.3 The OverWatch Watchtower

The Watchtower is an extra meta-workflow (not one of the 8 content agents) that monitors the crew:

- **Heartbeat sweep** every 15 min: checks each workflow's last successful execution time and each queue's oldest stuck row (status `processing` for > 45 min).
- **Quota radar**: counts Gemini/Groq requests used vs. daily limit from `meta_kpis`; warns before you hit 429s.
- **Success-rate tracker**: per-agent pass/fail ratios, rewrite counts, quarantine counts.
- **Alerts**: Telegram messages for stalls, failures, quota warnings, and daily digest at 07:30.
- **Kill switches**: config flags in `config` table — `publish_enabled`, `ingest_enabled`, `llm_enabled` — toggleable via Telegram commands (`/pause`, `/resume`, `/status`).

4. n8n Implementation Map

4.1 Workflow inventory



Figure 2 — One workflow per agent plus two cross-cutting workflows (Watchtower, Telegram bot).

Workflow	Trigger	Purpose
OW-01 scrape	Schedule: every 4 h	Pull 16 RSS feeds in parallel, normalize, hash-dedup, insert into headlines.
OW-02 score	Schedule: daily 06:00	Score unprocessed headlines, apply rubric + caps, write scored.
OW-03 write	Schedule: daily 06:30 (or after score)	Write ≥ 3,000-word articles in two 15-item batches (quota-paced).
OW-04 factcheck	Schedule: after each write batch	Verify claims, return verdict, route rewrites back to OW-03 (max 2).
OW-05 group	Schedule: after factcheck	Build group units + audit bundles, write groups.
OW-06 media	Schedule: after group	Generate art via Pollinations, render 10 cards with Pillow, write cards.
OW-07 caption	Schedule: after media	Generate caption + hashtags per group, length-gated, write captions.
OW-08 schedule	Schedule: every 15 min	Pick due posts, create 10 child + 1 parent carousel containers, publish, log permalink. v1.1: pre-loads next 1-2 days' slots via publish_time; enforces freshness expiry.
OW-09 watchtower	Schedule: every 15 min	Health sweep, quota radar, Telegram alerts, daily digest.
OW-10 telegram	Webhook (Telegram)	Commands: /status /pause /resume /run-cycle /digest.

0W-11 errors	n8n Error Trigger	Every workflow error → alert to Telegram (deduplicated).
0W-12 insights	Schedule: daily 08:00	Pull IG Insights (reach, engagement, carousel completion) into <code>meta_kpis</code> ; feeds daypart and caption tuning.
0W-13 review	Webhook (Telegram)	Human-review bridge: approve/reject drafts and decks from Telegram during the test phase.
0W-14 queue	Schedule: every 30 min + on score-complete	Queue Manager: re-rank pool by priority×freshness, expire stale candidates, assign slots, replace expired queued posts.

4.2 Credentials to configure once

Credential	Where	Notes
Gemini API key	n8n “Google Gemini” credential (or HTTP header)	From <code>aistudio.google.com/apikey</code> . Free tier, no card. Enable Gemini API in Google Cloud Console.
Groq API key	HTTP header credential	From <code>console.groq.com</code> . Free tier, no card.
Meta App credentials	HTTP query credential	App ID/Secret + long-lived Page token (60 days, refreshable) + IG user ID. See §12.3.
Telegram bot token + chat ID	n8n Telegram credential	From BotFather; chat ID from <code>/start</code> with your bot.
Pollinations	No credential needed (public URL)	No key; add <code>?nologo=true</code> to requests.

4.3 Module template for every agent

Every agent workflow follows the same 5-node skeleton, which keeps the system uniform and debuggable:

1. **Trigger** (Schedule or webhook) — when the agent wakes up.
2. **SQL Select** (Code node) — claim work: `SELECT ... WHERE status='pending' LIMIT n`.
3. **Process** (AI Agent / HTTP / Code) — the agent’s core work, with input JSON, strict output schema.
4. **Validate** (Code node) — schema check, length/quality gates, retry counter bump.
5. **SQL Write + state update** (Code node) — insert results, set `status='done'`, append to `audit_log`.

State machine used by all rows: `pending → processing → done`, with `processing → failed → retried(≤3) → rejected` on errors. Reasons are always written to `audit_log`.

Agent 1 — Scraper Agent

Collects 100+ news headlines per cycle from RSS feeds across topics

5.1 Feed library (16 feeds, all live & free)

Feed	URL	Items	Topics
Google News — Top Stories (EN)	<code>news.google.com/rss?hl=en-US&gl=US&ceid=US:en</code>	~25	Mixed
Google News — World	<code>news.google.com/rss?hl=en-US&gl=US&ceid=US:en&topic=w</code>	~25	World
Google News — Business	<code>news.google.com/rss?hl=en-US&gl=US&ceid=US:en&topic=b</code>	~25	Business
Google News — Technology	<code>news.google.com/rss?hl=en-US&gl=US&ceid=US:en&topic=t</code>	~25	Tech
Google News — Science	<code>news.google.com/rss?hl=en-US&gl=US&ceid=US:en&topic=snc</code>	~25	Science
Google News — Health	<code>news.google.com/rss?hl=en-US&gl=US&ceid=US:en&topic=m</code>	~25	Health
Google News — Sports	<code>news.google.com/rss?hl=en-US&gl=US&ceid=US:en&topic=s</code>	~25	Sports
Google News — Entertainment	<code>news.google.com/rss?hl=en-US&gl=US&ceid=US:en&topic=e</code>	~25	Culture
Google News search “artificial intelligence”	<code>news.google.com/rss/search?q=artificial+intelligence&hl=en-US</code>	~20	Tech
BBC News (World)	<code>feeds.bbc.co.uk/news/world/rss.xml</code>	~20	World
DW (Deutsche Welle)	<code>rss.dw.com/rdf/rss-en-all</code>	~20	World
Al Jazeera English	<code>aljazeera.com/xml/rss/all.xml</code>	~15	World
France 24 English	<code>france24.com/en/rss</code>	~15	World
TechCrunch	<code>techcrunch.com/feed/</code>	~20	Tech
The Verge	<code>theverge.com/rss/index.xml</code>	~15	Tech
Hacker News (front page)	<code>news.ycombinator.com/rss</code>	~30	Tech

All URLs verified reachable (HTTP 200) in August 2026. NPR’s feed requires a browser User-Agent; if you add it, set the header `User-Agent: Mozilla/5.0` in the RSS Read node. Expect 250–350 items raw; after dedup you comfortably exceed the 100-headline minimum. Add more feeds by cloning one RSS Read branch.

5.2 Workflow (0W-01 scrape)

1. **Schedule Trigger** — cron `0 */4 * * *` (every 4 h; six ingest windows/day).
2. **RSS Read ×16 in parallel** — one per feed. Set Options → Ignore SSL errors off; timeout 20 s; continue-on-fail so one dead feed never kills the cycle.
3. **Code: normalize** — for each item build the canonical record:

```
{
  "title":  clean(title),           // strip HTML tags, whitespace, truncate 140 chars
  "source": feed name,
  "url":    item link (canonical),
  "topic":  mapped category,       // world|tech|business|science|health|sports|culture
  "published_at": ISO8601,
  "fetched_at": now,
  "title_hash": md5(normalize(title)) // lowercase, strip punct/stopwords order
}
```

4. **Code: ingest-dedup** — keep only rows whose `title_hash` is not already in `headlines` (history kept 14 days).
5. **Code: insert** — batch INSERT with status `pending`; log counts to `audit_log` and `meta_kpis`.

5.3 Quotas & outputs

- **Minimum guarantee:** 100+ unique headlines per cycle by design — the 16 feeds deliver 250+ raw items, and dedup typically removes only 15–30% (cross-publishing agencies). A **coverage guard** fires a Telegram warning if fewer than 100 unique headlines land in a cycle.
- **Recency guard:** items older than 48 h are dropped at ingest (news decays fast; scoring rewards freshness).
- **Cross-cycle dedup (v1.1):** embedding cache keeps last 14 days of selected headlines; a candidate at cosine ≥ 0.85 vs. anything already published or pooled is excluded before scoring (stops the same story being re-picked daily from Google News RSS).
- **Output contract:** `headlines` rows with status `pending` → consumed by the Scoring Agent. Scraper itself calls zero LLMs — 100% quota-free.

Agent 2 — Scoring Agent

Consistent scoring rubric for maximum engagement; keeps 30–40, discards the rest

6.1 Workflow (OW-02 score)

1. **Schedule Trigger** — daily 06:00 (after the 04:00 ingest window has filled the queue).
2. **SQL Select** — unprocessed headlines (`status='pending'`), oldest first, max 200.
3. **Split In Batches** — 20 headlines per batch (fits context; keeps LLM call volume sane).
4. **AI Agent (Gemini 2.5 Flash)** — system prompt = rubric (Appendix B.1); output = strict JSON: one object per headline with 7 subscores, 0–10 each, plus 1-line rationale.
5. **Code: score & cap** — weighted total (formula below), apply topic cap, enforce 30–40 ceiling.
6. **Code: semantic dedup** — compare against already-selected headlines via embeddings (see 6.5).
7. **Code: write** — INSERT into `scored`; rejected rows → `audit_log` with reason (nothing is silently deleted).

6.2 The scoring rubric — 7 weighted engagement KPIs

#	KPI	Weight	What the scorer evaluates (0-10 anchors)
1	Click-bait / curiosity gap	20%	Does the headline create an information gap that demands a click? 0 = flat statement, 10 = irresistible question/contrast (“Why the world’s biggest company just reversed course”).
2	Emotional resonance	15%	Surprise, awe, outrage-avoidant interest, inspiration, fear-of-missing-out. 10 = universally felt emotion; 0 = dry bulletin. (Outrage bait scores down, not up — it burns the account.)
3	Practical relevance	15%	How directly does it affect the reader’s money, work, health, or daily life?
4	Freshness	15%	Age ≤ 6 h = 10; 6–24 h = 7; 24–48 h = 3; > 48 h dropped at ingest anyway.
5	Visual potential	15%	Can this story be told in 10 beautiful cards? Maps, photos, charts, people, places > abstract policy text.
6	Authority & verifiability	10%	Comes from a credible outlet; at least 2 independent sources likely to exist. Reduces fact-check failures later.
7	Shareability	10%	Would readers forward it to someone? Identity-relevant (“this is about people like me”), conversational, or “must-know” framing.

Score formula: `total = 0.20·s1 + 0.15·s2 + 0.15·s3 + 0.15·s4 + 0.15·s5 + 0.10·s6 + 0.10·s7` → 0–100.

6.3 Pass rules (deterministic, applied after the LLM scores)

- **Threshold:** total ≥ 72.
- **Topic cap:** maximum 8 selected headlines per topic (world/tech/business/science/health/sports/culture) — guarantees a mixed feed, which Instagram’s algorithm and followers both reward.
- **Ceiling:** keep at most 40 per cycle, rank-ordered; if more pass, take the top 40.
- **Candidate pool (v1.1):** the 30–40 winners do not go straight to publishing — they enter the **pool** table with priority = total score and a freshness timestamp. The Queue Manager (\$15, OW-14) publishes from the pool at 3 posts/day, so a burst of 40 doesn’t create a 2-week backlog of stale news.
- **Floor:** keep at least 30; if fewer pass, take top scorers down to 72-threshold relaxed to 65 once, then top-30 regardless (channel must never starve).
- **Spread bonus:** +2 points if the story is from a non-English-region outlet (international angle is a differentiator).
- **Tie-break:** higher freshness wins, then higher visual potential.

6.4 Deduplication (two layers)

- **Layer 1 — exact hash:** MD5 of the normalized title (already applied at ingest).

- **Layer 2 — semantic:** embedding of each candidate headline compared with already-selected ones; cosine similarity $\geq 0.85 \rightarrow$ duplicate (keep the higher-scored one, log the merge). Embeddings run free on the Oracle VM via `sentence-transformers` (all-MiniLM-L6-v2) or the free Gemini embedding endpoint.

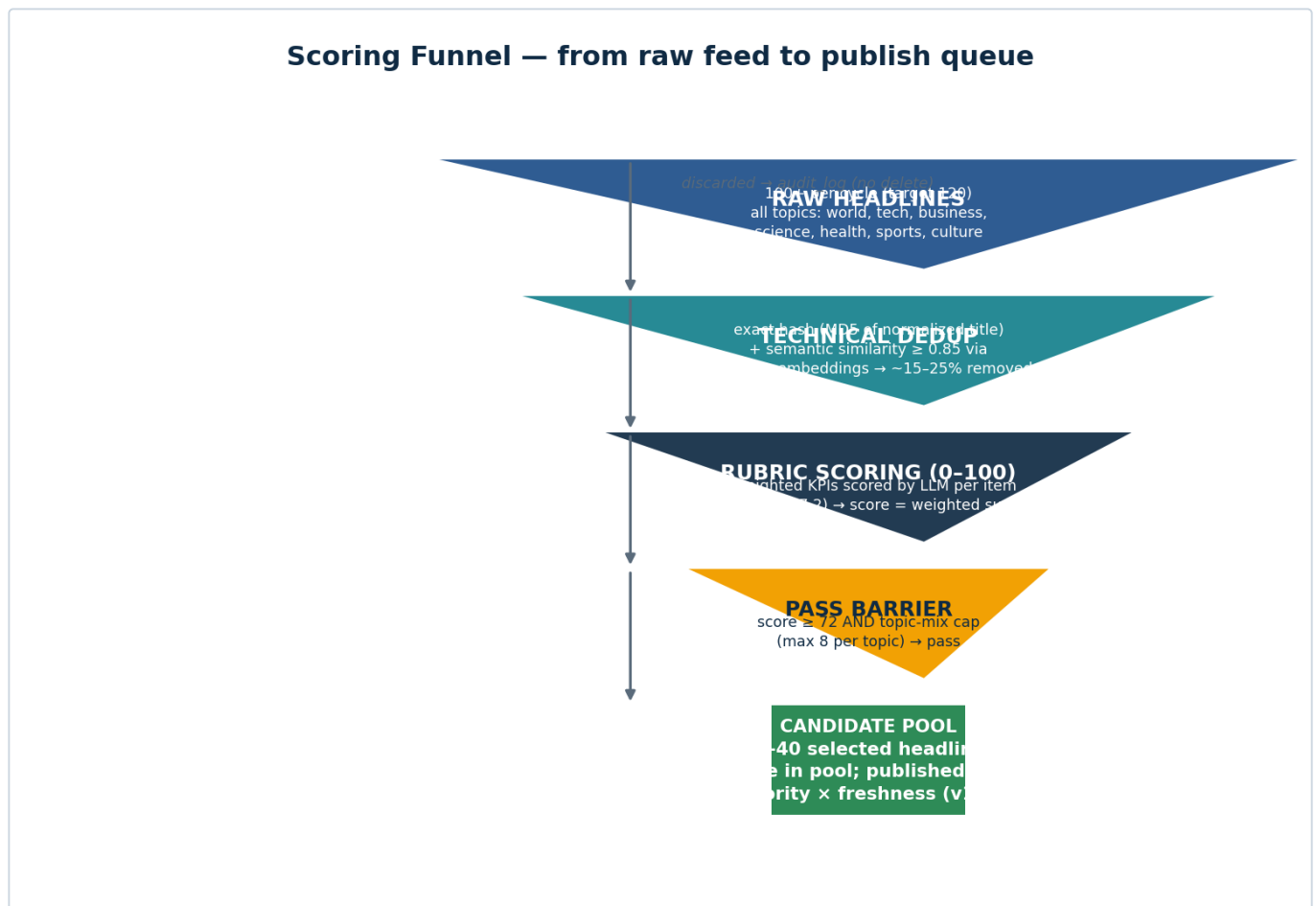


Figure 3 — The scoring funnel. ~120 raw $\rightarrow$ ~100 unique $\rightarrow$ ~50 above threshold $\rightarrow$ 30-40 after topic caps.

6.5 Output contract

```

{
  "headline_id": "H-20260825-0042",
  "title": "...", "url": "...", "topic": "tech",
  "subscores": {"curiosity": 8, "emotion": 7, "relevance": 9,
    "freshness": 10, "visual": 6, "authority": 8, "shareability": 7},
  "total_score": 79.5,
  "passed": true,
  "rank": 3
}
  
```

Consistency rule: the rubric is fixed in the prompt (Appendix B.1) and versioned in the `config` table. Any rubric change bumps its version, and historical scores stay comparable because every scored row stores the rubric version used. Re-scoring the same headline with the same version yields the same result because the LLM runs at temperature 0.

Agent 3 — Content-Writing Agent

Writes a 3,000+ word, attention-grabbing, engagement-maximizing article per selected headline

7.1 Workflow (0W-03 write)

1. **Trigger** — schedule 06:30, or an n8n hook fired when scoring finishes.
2. **SQL Select** — scored-and-passed headlines without an article (`status='pending'`), ordered by rank.
3. **Batch gate** — process in two batches of ~15 (quota pacing, \$7.6).
4. **Per article:**
 1. **Research call (1 Gemini request):** given headline + source URL, fetch the source article text (HTTP Request), produce an outline: angle, 8 section titles, key facts, data points, quotes to verify.
 2. **Section calls (8 Gemini requests):** one per section, 400–500 words each, with the outline + previously written sections as context (keeps continuity and stays inside output-token limits).
 3. **Assembly (Code):** concatenate sections into one markdown document; add headline (H1), standfirst, byline block, fact-check note, and sources list.
 4. **Hard gate (Code):** word count $\geq$ 3,000 (target 3,200–3,500). If short, the missing section is regenerated once with an “expand” instruction.
5. **SQL write** — INSERT `articles` with `status='pending_review'` (awaiting fact-check), `word_count`, `draft_rev`.

7.2 Article anatomy (engagement-engineered)

Section	Word budget	Engagement function
H1 headline + standfirst	60–80	Curiosity gap; 1-sentence promise of what the reader gains.
1. The hook	250–300	Open with a concrete scene, number, or contradiction — never a definition. First 3 sentences decide the swipe.
2. What happened (the facts)	450–500	Who/what/when/where/why in plain language; named entities; exact numbers.
3. Why it matters	400–450	Reader impact: money, health, work, society. The “so what” is explicit.
4. Context & background	450–500	History, prior events, key players — makes the story understandable alone.
5. Different views / reactions	350–400	Quotes and positions from at least two sides; balanced, no strawmen.
6. What happens next	300–350	Timeline, expected moves, open questions — feeds comment sections.
7. Data & figures	300–350	Stats box, mini-table or list of key numbers (visual-friendly for cards).
8. Bottom line + CTA	200–250	Summary in 3 bullets + engagement CTA (“What would you do?” type question).
Sources footer	—	Numbered links; every factual claim maps to a source (fact-checker uses this).

7.3 Style rules (in the system prompt)

- Reading level: grade 8–10 (Flesch 60+). Short sentences. One idea per paragraph.
- Every 3–5 paragraphs: a subheading, a bullet list, or a bolded takeaway — scannability is retention.
- Numbers over adjectives; named people over institutions; active voice.
- No invented quotes, no invented stats, no speculation presented as fact. Anything uncertain is explicitly labeled.
- Neutral tone with a clear point of view; never insult any side of a story.
- Write for a global English-speaking audience; explain jargon on first use.

7.4 Quota math (why 40 articles fit the free tier)

Workload	Gemini requests	Notes
Research/outline per article	1	$\times 40 = 40$
Sections per article	8	$\times 40 = 320$
Expansion/rewrite allowance (10%)	~36	Word-count gate + fact-check rewrites
Fact-checker calls (Agent 4)	~80	claim extraction + verdict per article
Scoring + captions + misc	~25	small prompts, can run on Groq instead
Total per 48 h cycle	~500	~250/day

Headroom strategy (v1.1): if Gemini’s free tier is exhausted or degraded, every text task fails over automatically: Gemini → Groq (free, ~30 RPM) → local Ollama on the VM (unlimited). The Watchtower health flag tracks provider availability. Gemini free tier with OAuth = 1,000–1,500 req/day (well above ~250/day); with a plain API key = ~250/day — still workable if the writer runs 15 articles per day over 3 days, or if short-output tasks (scoring, captions, verdicts) run on Groq’s free tier instead. The batch gate in 7.1 exists exactly for this: batches never exceed 15 per run, and a 429 simply postpones the next batch. No cost either way.

Agent 4 — Fact-Checker Agent

Verifies every article against credible sources; too much false information → rewrite

8.1 Workflow (OW-04 factcheck)

1. **Trigger** — when a write batch completes.
2. **SQL Select** — articles with `status='pending_review'` and `draft_rev ≤ 2`.
3. **Code: claim extraction (Gemini)** — from the article, extract the list of verifiable factual claims (max 12): numbers, dates, names, attributions, causal statements. Output JSON:

```
[{"claim": "GDP grew 3.1% in Q2", "type": "statistic", "importance": "high"}, {"claim": "Company X acquired Y for $4.7B", "type": "event", "importance": "high"}]
```
4. **Per claim — evidence gathering (HTTP, zero LLM):**
 - Fetch the original source article (from the group's `url`).
 - Search Google News RSS: `news.google.com/rss/search?q=<claim keywords>&hl=en-US` → top 5 results; fetch the 2–3 most authoritative (Reuters, AP, BBC, outlet home sites).
 - Optional free augment: Brave Search API (1 free query/s) for non-news verification.
5. **Code: verdict (Gemini)** — one JSON verdict per claim: `supported | unsupported | contradicted | unverifiable` + confidence 0–1 + short evidence note.
6. **Code: aggregate:**

```
pass_ratio = supported / total_claims
score = pass_ratio * 0.7 + mean_confidence * 0.3
```

 - **PASS** if $\text{score} \geq 0.90$ and no `contradicted` claim with `importance=high`.
 - **REWRITE** if $0.75 \leq \text{score} < 0.90$, or any high-importance claim is `unsupported` / `contradicted`. The article returns to the Writer with the list of failed claims; writer `rev ≤ 2`.
 - **QUARANTINE** if $\text{score} < 0.75$ or the 3rd draft still fails — the group is shelved with a reason; never published.
7. **Code: write** — INSERT into `fact_checks` (claims, sources checked, verdict, confidence, notes) and update article status. Every check lands in `audit_log`.

8.2 Credibility rules for sources

- **High trust:** Reuters, AP, AFP, BBC, DW, Al Jazeera, Guardian, NPR, government/agency pages, primary documents, academic journals, company press releases (labeled as such).
- **Medium trust:** major national dailies, trade press. **Low/never:** anonymous blogs, aggregators, social posts, AI-content farms, sites with no editorial masthead.
- Two independent high-trust sources → claim considered corroborated even if the original outlet differs.
- **Auto-flag topics** (stricter bar): health/medical claims, financial numbers, election results, crime allegations, anything involving minors. Any contradicted claim in these areas forces REWRITE regardless of score.

Error budget: with a 72-threshold rubric that already favors authoritative sources, expect ~15–25% of drafts to need one rewrite and < 5% to quarantine. Rewrite feedback is structured JSON, so the writer fixes exactly the failed claims — it never rewrites from scratch.

Agent 5 — Grouping Tool

Bundles each headline + article into one auditable group unit

9.1 What a group is

A **group** is the atomic unit of the channel: one headline + one verified article + all metadata produced along the way (scores, fact-check record, versions) + the eventual cards and caption. Groups are what get audited, what the media agent consumes, and what the scheduler publishes.

9.2 Workflow (0W-05 group)

1. **SQL Select** — verified articles (`status='verified'`) without a group.
2. **Code: assemble** — build the group record with audit ID `0W-YYYYMMDD-NN` (cycle date + sequence) and snapshot all upstream rows (headline, scores, article, fact-check summary).
3. **Code: audit bundle** — write `/audit/0W-.../bundle.json` (machine-readable) and a human-readable `bundle.html` /PDF export for manual review.
4. **SQL write** — INSERT `groups`, status `ready_for_media`.

9.3 Audit bundle example (excerpt)

```
{
  "audit_id": "0W-20260825-07",
  "headline": { "title": "...", "url": "...", "source": "BBC", "topic": "world" },
  "score": { "total": 81.0, "rubric_version": "v1.0", "subscores": { ... } },
  "article": { "id": "A-...", "word_count": 3264, "draft_rev": 1 },
  "fact_check": { "verdict": "PASS", "score": 0.94, "claims": 11,
    "sources": ["reuters.com/...", "apnews.com/..."] },
  "cards": [{ "card_no": 1, "img": "cards/0W-20260825-07/c1.png" }],
  "caption": { "variant": "A", "chars": 1450 },
  "post": { "status": "queued", "publish_at": "2026-08-27T18:30:00+05:00" },
  "timeline": [ { "stage": "scrape", "at": "...", "duration_ms": 3100 } ]
}
```

Agent 6 — Media-Generating Agent

10 themed cards per group: headline + complete article, strict theme

10.1 The theme system (strict, pre-selected)

The theme is enforced by a **design token file** (`/data/theme.json`) that the render code reads for every card — nothing is left to the image model's taste. Two built-in themes; you can add more, but the channel publishes under exactly one active theme at a time (`config.theme`).

Token	Theme A “NewsBlue” (default)	Theme B “EditorialLight”
Palette	Navy #0F2A43, Blue #1D4E89, Accent amber #F2A104, Paper #F4F7FB	Ink #1B1B1B, Paper #FAF6EF, Accent red #C02037, Grey #6E6A63
Type	DejaVu Sans Bold (headlines), regular (body)	DejaVu Serif (headlines), sans (body)
Layout grid	Margins 72 px; header band 96 px with logo + issue tag; safe zones 200 px bottom, 140 px sides	Margins 84 px; top rule line; drop-cap style opener
Artwork	AI background at 30% opacity under a navy overlay (text stays readable)	Full-bleed art on cards 1 and 10 only; typographic cards in between
Masthead	“OVERWATCH · DAILY BRIEF · No. NN” top-left; card number bottom-right	“OVERWATCH — Issue NN” top-center
CTA card (10)	Follow / share prompts, channel handle, next-issue teaser	Same, serif styling

10.2 The 10-card plan (per group)

Card	Content	Purpose
1	Full-bleed AI artwork + headline (max 2 lines) + issue tag	Stop the scroll — the thumbnail.
2	Standfirst + “the 5 key facts” list	Instant value; completes the hook.
3–9	Article in dense-zoom blocks: ~340 words per card in 44–48 px type	Carries the complete 3,000+ word article on 7 dense cards + cards 2/10 (requirement: full article on the cards; users pinch-zoom to read).
10	CTA card: summary bullets, follow/share prompt, sources line, caption teaser	Conversion: saves, shares, follows.
10	CTA card: summary bullets, follow/share prompt, sources line, caption teaser	Conversion: saves, shares, follows.

10.3 Artwork generation (Pollinations, no key)

- **Endpoint:** `https://image.pollinations.ai/prompt/{urlencoded prompt}?width=1080&height=1350&nologo=true&model=flux` (GET).
- **Prompt template:** `"Editorial magazine background art, {topic} themed, {scene from headline}, muted {palette} colors, no text, no letters, no words, soft depth of field, 4:5"`. The “no text” clauses are mandatory — AI-drawn letters look broken; all typography is added by Pillow.
- **Fallback chain:** Pollinations fails → retry 2× with 10 s delay → deterministic gradient + geometric pattern via Pillow (still on-theme). A card never goes missing.

10.4 Rendering (Pillow) — the deterministic half

- Canvas 1080×1350 px (4:5, Instagram's preferred ratio), PNG, sRGB.

- Text engine (v1.1 dense-zoom typography): word-wrap by character measure; headline sizes 64–88 px; body 44–48 px with 1.3 line height — up to ~340 words per card. This is the standard “zoom to read” pattern used by news pages on Instagram; readability is proven in the phone test (§17.3) before auto-publish is enabled. Minimum 8 words visible per screen on card 1.
- Safe zones respected: 140 px side margins, 200 px bottom (UI overlays hide content there).
- Accessibility: contrast check (WCAG AA) between text and card background; adjust overlay opacity automatically.
- Card QA gate: every deck passes the pixel/count/word-coverage check; a 3-deck **phone test** (deck opened on a real phone) is mandatory in the 30-day plan before auto-publish.
- Output: `/media/OW-yyyyymmdd-NN/card-01.png ... card-10.png`; each row logged to `cards`.

10.5 Workflow (`OW-06 media`)

1. SQL Select groups with `status='ready_for_media'`.
2. Code: chunk the verified article into the 10-card content plan (9.2 mapping).
3. HTTP Request ×10 (parallel): Pollinations art for card 1 (and 10 in Theme B).
4. Execute Command node: `python3 render_cards.py --group OW-... --theme NewsBlue` (script lives on the VM, reads `theme.json`).
5. Validation: 10 files exist, each ≥ 200 KB and 1080×1350; else re-render once.
6. **Upload (v1.1)**: push the 10 PNGs to a Cloudflare R2 bucket via its S3-compatible API and store the public HTTPS URLs in `cards` — Instagram requires public image URLs, which the ephemeral n8n tunnel cannot guarantee (§12.3). R2 free tier: 10 GB storage, no egress fees.
7. SQL write → `status='ready_for_caption'`.

Why not let the AI generate whole cards? Image models can’t typeset 3,000 words reliably. Split design: AI supplies evocative backgrounds; Pillow supplies deterministic, theme-strict typography. This is what keeps 400 cards per cycle consistent at \$0.

Agent 7 — Caption-Generator

Views the finished post and writes a suitable, engagement-maximizing caption

11.1 Workflow (`OW-07 caption`)

1. SQL Select groups with `status='ready_for_caption'` (cards exist).
2. **AI Agent (Gemini Flash or Groq)** — input: headline, standfirst, article summary (first 300 words), key figures, card-1 text, theme. Output: caption JSON with fields below.
3. **Code: gate** — caption $\leq$ 2,200 characters (Instagram hard limit: 2,200; keep 2,000 for safety), 3-8 hashtags, contains exactly one CTA. Fail → one regeneration.
4. **Code: write** — two variants A/B for testing; primary goes to `captions`.

11.2 Caption anatomy

Block	Rule
Hook (1-2 lines)	Starts with the story's most surprising fact or a direct question. Never starts with "Breaking:".
Summary (3-5 lines)	Standfirst rewritten for voice; key numbers included.
Value line	"Swipe through for the full story + the 5 key facts" — drives carousel completion (a top ranking signal).
CTA (exactly 1)	Rotates per post: question ("What's your take?") / save ("Save this for later") / follow ("Follow for daily briefs").
Hashtags	3-8, mix: 1 broad (#News), 1 topic (#Technology), 1 niche (#AIBrief), 1 branded (#OverWatchDaily). All channel-level, never spam-stuffed.
Sources line	"Sources: Reuters, AP, BBC — verified." (transparency = trust = retention).

Agent 8 — Scheduling Agent

Publishes each post at a suitable time with pre-selected posting details

12.1 Workflow (OW-08 schedule)

1. **Schedule Trigger** — every 15 min, all day.
2. **SQL Select** — posts where `status='queued'` AND `publish_at ≤ now()`, order by `publish_at`, limit 1 (one API batch at a time).
3. **Assignment step (when a group is ready):**
 - Compute next free slot from the daypart table (12.2) and existing `posts` rows.
 - Rules: max 3 posts/day; minimum 3 h between posts; no post between 23:00–06:00 local audience time; slot chosen by daypart weight, tie → earlier date.
4. **Publish sequence (Figure 6):** create 10 child image containers → create parent CAROUSEL container with children list → `media_publish` → fetch permalink → Telegram confirmation.
5. **API-native scheduling (preferred in v1.1):** the Graph API accepts `publish_time` (unix ts) on container creation for 15 min–24 h ahead. Every morning the scheduler pre-loads the next 1–2 days' slots into Meta, so publishing continues even if the VM is down; only immediately-due posts are published synchronously.
6. **Code: write** — update `posts` status `published`, store `media_id` + permalink.

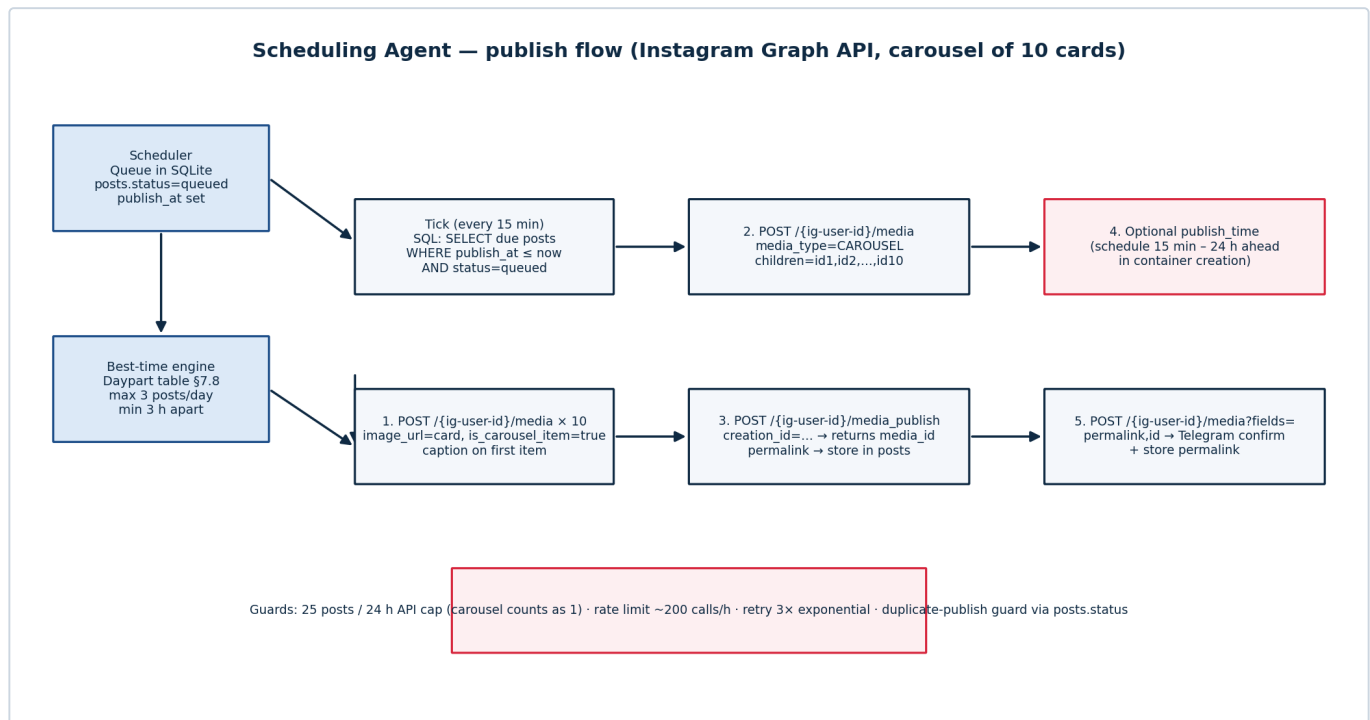


Figure 6 — Instagram Graph API carousel publish flow (all free).

12.2 Best-time engine (daypart model)

Daypart (audience local time)	Weight	Rationale
06:00–10:00	5%	Morning commute; low but real.
10:00–13:00	15%	Mid-morning scroll break.
13:00–17:00	20%	Lunch + afternoon lulls.
17:00–21:00	45%	Prime time: after work, commute, dinner — peak feed activity.
21:00–23:00	15%	Evening wind-down.

Timezone strategy: publish in the audience's local time. Start with your home audience (Asia/Karachi, UTC+5); when Insights shows a second dominant region, add a second slot map — the scheduler reads `config.timezone` and picks slots per audience cluster. Slots are computed as 17:00/18:00/19:00–20:30 local, capped at 3/day.

12.3 Instagram Graph API — prerequisites and exact calls

Hard requirements (all free, all take time — start week 1):

- Instagram account switched to **Business or Creator** (free, in-app).
- A **Facebook Page** linked to that Instagram account (free; required by Meta for API access).
- A **Meta Developer app** (free at developers.facebook.com) with `instagram_basic` and `instagram_content_publish` permissions.
- **App Review** approval for `instagram_content_publish` (free; can take days-weeks; you can test with your own account in development mode meanwhile — your account must hold the Admin/Test role).
- Long-lived token: exchange short-lived user token → long-lived (60 days) via `/oauth/access_token?grant_type=fb_exchange_token`; re-refresh before expiry (build a monthly reminder into the Watchtower).
- **Public media URLs (v1.1):** every `image_url` must be reachable over public HTTPS. The n8n tunnel URL changes on restart, so all card images are uploaded to a free Cloudflare R2 bucket (stable public URL, no egress fees) and served from there (\$10.5).

```
# 1) create each child image container (card 1..10)
POST /{ig-user-id}/media
  image_url = https://host/media/OW-.../card-01.png
  is_carousel_item = true
  caption = (only on first item; ≤ 2200 chars)

# 2) create the carousel container
POST /{ig-user-id}/media
  media_type = CAROUSEL
  children = [id1, id2, ..., id10]

# 3) publish
POST /{ig-user-id}/media_publish  creation_id = <container id>
  → media_id

# 4) fetch permalink
GET /{media_id}?fields=permalink,id

# optional: schedule instead of immediate publish
add "publish_time" = unix timestamp (now+15min .. now+24h) to step 1&2
```

12.4 Guards and limits

- **Platform cap:** 25 published posts per 24 h per account (a carousel counts as 1). OverWatch posts 3/day — 8× headroom.
- **Rate limit:** ~200 API calls per hour per user token; our flow uses 22 calls per post (10+1+1+... with retries) — 3 posts = ~70 calls/day, safe.
- **Duplicate guard:** `posts.status` prevents double-publishing a group; idempotency check on container creation errors.
- **Failure handling:** 3 retries with exponential backoff; on final failure → Telegram alert, post marked `failed`, slot freed for a later slot; nothing is lost from the queue.
- **Grace period:** if `publish_time` scheduling was used, a daily reconcile step re-checks any post still `queued` past its slot and republishes immediately.
- **Freshness expiry (v1.1):** a queued post whose headline is older than 72 h is dropped (`posts.status='expired'`) and the Queue Manager promotes the highest-priority pool candidate into its slot — stale news can never sit in the queue.

13. Shared Storage & Data Model

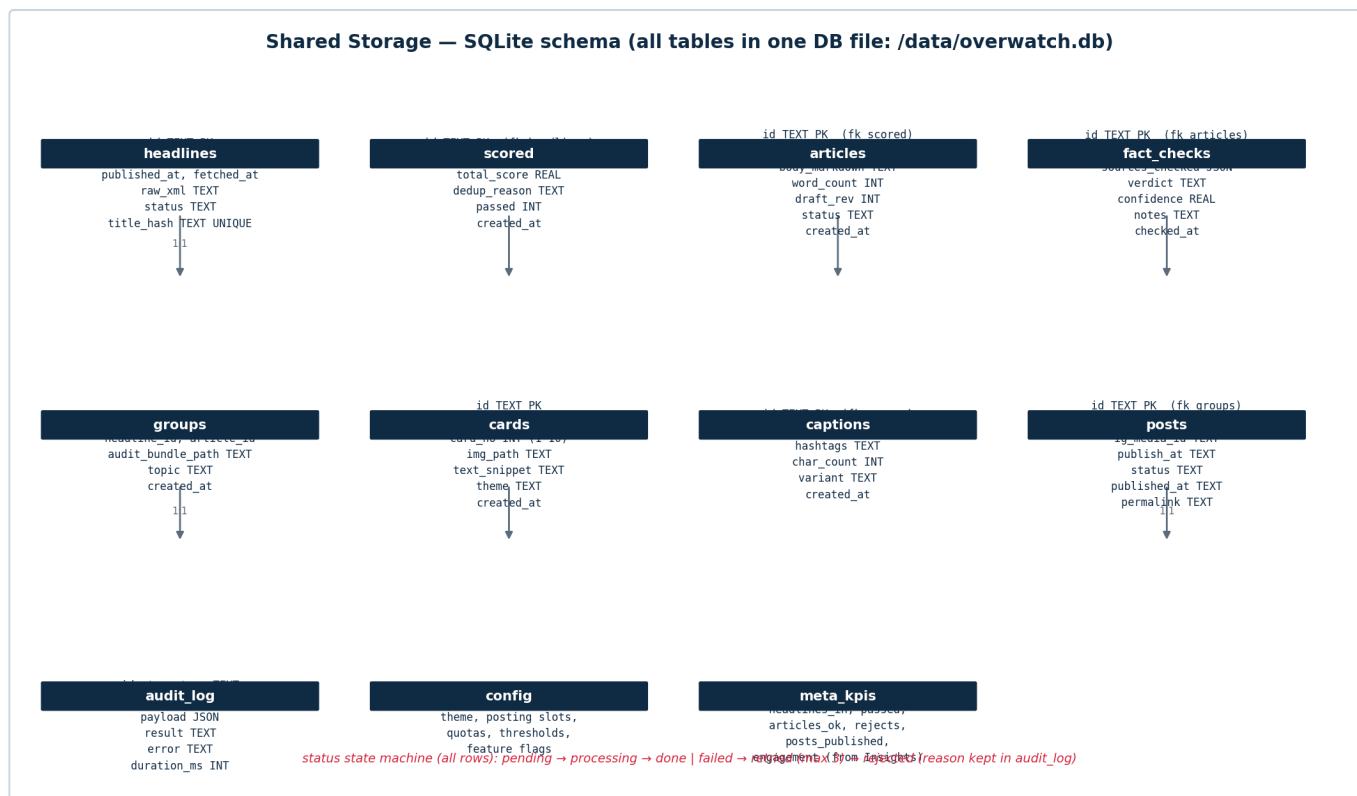


Figure 4 — SQLite schema, one file `/data/overwatch.db`. Every table is written by one agent and read by the next; `audit_log` and `meta_kpis` are written by all.

13.1 Access rules

- n8n Code nodes connect via `sqlite3` (Python) / `better-sqlite3` (JS) with `WAL` mode enabled for concurrent reads/writes.
- Writes are transactional: insert + status update in one `BEGIN/COMMIT`.
- Status changes are the only way stages consume work (no cross-workflow JSON hand-offs — this is what makes every agent’s “shared access” requirement true).
- Backups: nightly `sqlite3 .backup` to `/backups/`, retained 14 days, synced to free cloud storage (Google Drive via rclone free tier) weekly.
- Media objects: rendered cards live in `/media` and are mirrored to the free Cloudflare R2 bucket (public HTTPS URLs for Instagram); the DB stores only the URL.
- Pool & queue tables (v1.1): `pool` (candidates, priority, freshness, expiry) and `posts` doubles as the publish queue (status + `publish_at`) — managed by OW-14.

14. Watchtower, Alerts & Monitoring

14.1 Health checks (every 15 min)

Check	Alert if...	Severity
Workflow last-run age	Any content workflow silent > 24 h	High
Stuck rows	status='processing' for > 45 min	High
Ingest coverage	< 100 unique headlines in last cycle	Medium
Quota radar	Gemini/Groq usage > 80% of daily free limit	Medium
Fact-check pass ratio	Quarantine rate > 15% (rubric drift)	Medium
Publish success	Any post failed 3x or publish queue > 48 h old	High
Token age	Meta long-lived token < 7 days to expiry	High
Disk/DB size	disk > 85% or db > 2 GB	Low

14.2 Telegram bot commands

Command	Action
<code>/status</code>	Pipeline summary: headlines in, scored, articles, verified, groups, cards, queued, published today.
<code>/run-cycle</code>	Manually triggers the full cycle (or a stage: <code>/run-cycle score</code>).
<code>/pause /resume</code>	Global or per-stage kill switch via config flags.
<code>/digest</code>	Forces the daily digest (07:30 default).
<code>/quota</code>	Current LLM usage vs free limits.

14.3 Daily digest (07:30)

Telegram message with: cycle summary numbers, today’s publish plan (times + headlines), yesterday’s published posts with permalinks, quarantine list (if any), quota usage, and a one-line health verdict. A **human-approval flag** (`config.auto_publish`) can require a “go” during the test phase and switch to full-auto later.

15. End-to-End Workflow — the 40-step runbook

One cycle in full, with owners and timing. Steps repeat per batch where marked “(per article)”.

- ❶ **Scrape (00:00-02:00):** Trigger every 4 h; pull 16 RSS feeds in parallel (continue-on-fail).
- ❷ Normalize items → canonical records; compute title hashes; drop >48 h old.
- ❸ Dedup vs 14-day history; insert ≥ 100 unique headlines; coverage guard warns if < 100 .
- ❹ **Score (06:00):** Select pending headlines; batch of 20; call Gemini with rubric prompt (temp 0).
- ❺ Compute weighted scores; semantic dedup ($\cos \geq 0.85$); apply topic caps (8/topic).
- ❻ Keep top 30–40 (threshold 72; floor/ceiling rules); log discards with reasons.
- ❼ **Write batch 1 (06:30-17:30):** Take 15 ranked headlines; per headline: fetch source article (HTTP).
- ❽ (per article) Research call → outline JSON (angle, 8 sections, facts, quotes to verify).
- ❾ (per article) 8 section calls, 400–500 words each, sequential with context.
- ❿ (per article) Assemble markdown; apply style pass; word-count gate $\geq 3,000$.
- ⓫ (per article) Insert article status `pending_review`; log to audit.
- ⓬ **Fact-check batch 1 (17:30-23:30):** per article: extract claims (Gemini, JSON, max 12).
- ⓭ (per claim) Fetch original source + Google News RSS search + top 2–3 high-trust articles (HTTP, free).
- ⓮ (per article) Verdict call (Gemini): supported/unsupported/contradicted/unverifiable + confidence.
- ⓯ Aggregate: PASS / REWRITE / QUARANTINE per thresholds in §8.1; write `fact_checks`.
- ⓰ Rewrites: send failed-claims JSON to writer ($\text{rev} \leq 2$); rewrites slot into next writer batch.
- ⓱ **Write batch 2 (next day 06:30-17:30):** next 15 headlines + any rewrites from batch 1.
- ⓲ **Fact-check batch 2 (next day 17:30-23:30):** verify as above; update statuses.
- ⓳ **Group (23:45):** verified articles → group units with audit IDs; write audit bundles to /audit.
- ⓴ **Media (00:00-07:00 day 3):** chunk article into 10-card plan; Pollinations art (10 parallel GETs).
- ⓵ (per group) Pillow render 10 cards 1080×1350 per active theme; validate size/count; write cards.
- ⓶ (per group) Upload deck to Cloudflare R2; store public HTTPS card URLs in `cards`.
- ⓷ **Caption (07:00-09:00):** per group: caption generation (2 variants), gate $\leq 2,200$ chars + hashtag rules.
- ⓸ **Schedule (09:00):** Queue Manager (OW-14) picks top pool candidates (priority × freshness decay); assign slots: daypart weights, max 3/day, ≥ 3 h apart, audience timezone.
- ⓹ **Publish (rolling):** every 15 min tick picks due posts; create 10 child containers.
- ⓺ Create CAROUSEL parent container (children list); optional `publish_time` pre-scheduling.
- ⓻ `media_publish` → `media_id`; fetch permalink; update posts; Telegram confirm.
- ⓼ Daily reconcile: missed queued posts republished at next slot; token-age check; Insights pull (OW-12); pool expiry sweep (OW-14).
- ⓽ **Watchtower (continuous):** health sweep every 15 min; alerts; quota radar; digest 07:30.
- ⓾ Human review loop (test phase): approve daily digest; toggle `auto_publish` flag.



Figure 5 — 48-hour build timeline. Two writer batches deliberately span two days to stay inside Gemini’s free daily quota while never idling.

16. Phase 2 Roadmap (still \$0)

Upgrade	What changes	When
Facebook page posting	Reuse the same group/caption pipeline; add a Facebook Pages API branch (also free) — same deck, Facebook-native caption.	After IG stabilizes
Newsletter	Buttondown (free plan, unlimited subscribers ≤ 100) or self-hosted Listmonk on the VM — email the article digest weekly; n8n sends via SMTP.	Month 2
Reels previews	Pillow/ffmpeg (free) animates the 10 cards into a 15 s Reel — Reels reach > carousels on IG.	Month 2
Engagement loop	Graph API Insights pull each morning → auto-tunes daypart weights and caption CTA rotation.	Month 2
Second language	Google News RSS per language + Gemini translation pass; regional IG accounts.	Month 3
Video agent	Pollinations + ffmpeg free-tier video (or Peertube/Tumblr mirror) when the channel outgrows IG alone.	Month 3+
Community	Telegram channel auto-mirror of posts (Telegram API is free) — owned audience, zero platform risk.	Month 1
Queue analytics	Pool-health dashboard (median age at publish, discard reasons, topic mix) from <code>meta_kpis</code> — free, built into the Watchtower digest.	Month 2

17. Quality, Tests & Reliability (new in v1.1)

17.1 Why this section exists

A first-principles review of v1.0 found four structural gaps: (a) the 30–40-per-cycle output rate collides with the 3-posts/day publishing rhythm, creating an unbounded stale backlog; (b) the 10-card plan could not physically carry a 3,000-word article at readable type sizes; (c) Instagram requires public HTTPS media URLs, which the ephemeral n8n tunnel cannot reliably provide; (d) the plan relied on a single LLM provider. v1.1 closes all four and adds automated quality control around the LLM-dependent stages.

17.2 Queue Manager (OW-14) — the pool → queue engine

Element	Specification
Candidate pool	<code>pool</code> table: <code>headline_id</code> , <code>total_score</code> , <code>priority</code> (= score), <code>created_at</code> , <code>freshness_ts</code> (headline publish time), <code>status</code> . 30–40 rows per cycle.
Re-ranking	Effective priority = <code>total_score</code> × <code>freshness_decay</code> , where decay = 1.0 (0–12 h) → 0.75 (12–24 h) → 0.5 (24–48 h) → 0.25 (48–72 h) → 0 (expired). Re-run at every OW-14 tick.
Slot filling	Each tick: take the highest effective-priority candidate without a slot; assign the earliest free slot respecting daypart weights, max 3/day, ≥ 3 h apart, audience timezone.
Expiry	Candidate older than 72 h → status <code>expired</code> , reason logged. Pool is capped at 80 rows; overflow ages out oldest-first (nothing is deleted, everything is audited).
Replacement	If a queued post fails permanently or expires, its slot is re-filled from the pool at the next tick — the channel never has an empty slot.
Idempotency	One slot per candidate enforced by a UNIQUE constraint; re-runs of OW-14 are no-ops.

17.3 Automated quality gates & tests

Test	Runs when	What it catches
Scoring golden set	After any rubric/prompt change	40 curated headlines with expected pass/rank; score drift > ±5 points or flipped decisions fail the build.
Fact-checker golden set	After any fact-check prompt change	20 articles with planted errors; must flag ≥ 95% of planted errors and 0 false rejections of clean articles.
Writer word-count gate	Every article	≥ 3,000 words; ≥ 8 sections; no placeholder text (“[insert]”, “lorem”).
Card contract test	Every deck	10 files, 1080×1350, ≥ 200 KB, theme tokens present, word coverage ≥ 95% of article body.
Phone test	3 decks before go-live + monthly	Zoom-readability and swipe flow on a real phone (manual, 5 min).
Contract tests	On every workflow import	Sample JSON in/out per stage (scored → article → verdict → deck → caption → containers).
Integration dry run	Before every go-live	2 full cycles, auto-publish OFF, all statuses must reach <code>done</code> .

17.4 Human-review bridge (OW-13, test phase only)

- Queue of pending decks/captions; reviewer (you) gets one Telegram message per group with deck preview link + caption; reply `ok` or `reject`.
- Rejected groups return to the writer/caption stage with your note; nothing publishes without approval while `config.review_mode=on`.
- After 14 consecutive days of ≥ 95% approvals, switch `review_mode=off` (the digest remains the daily human checkpoint).

17.5 LLM failover matrix

Task	Primary	Fallback 1	Fallback 2
Scoring, verdicts, captions (short JSON)	Gemini Flash (free)	Groq (free)	Ollama local (free)
Article sections (long text)	Gemini Flash (free)	Gemini Flash-Lite (free)	Ollama local (free)
Embeddings (dedup)	Local sentence-transformers	Gemini embeddings (free)	—

The Watchtower pings each provider every 15 min (cheap health call); on failure it flips `config.llm_primary` and alerts Telegram. 429s trigger exponential backoff, never a crash.

17.6 Security hardening checklist (v1.1)

- n8n behind Cloudflare Tunnel with **basic auth + strong password**; admin UI never exposed on a public IP.
- Secrets in n8n credentials vault only — never in workflow JSON, prompts, or the DB.
- SQLite Code nodes use **parameterized queries only** (no string-concatenated SQL).
- Telegram bot webhook validates the secret token; commands allow-list only (no free-form shell).
- Card uploads to R2 use a scoped API token (write-only bucket).
- RSS URLs allow-listed; HTTP nodes set timeouts and max response sizes.
- Nightly DB backup is encrypted at rest (zip + password) before cloud sync.
- n8n Docker image version-pinned; weekly `docker compose pull` + test in a staging copy before switching.

17.7 v1.1 change log (summary)

Change	Location in this document
Candidate pool + Queue Manager (OW-14)	§6.3, §15, §17.2
Dense zoom typography (10-card fit for 3,000 words)	§10.2, §10.4, §17.3
Cloudflare R2 media hosting (public URLs)	§10.5, §12.3, §13.1
LLM failover matrix + provider health flag	§7.4, §17.5
Golden-set regression tests (rubric + fact-checker)	§17.3, §18
Human-review bridge (OW-13)	§4.1, §17.4
Insights feedback loop (OW-12) → daypart/caption tuning	§4.1, §16, §19
Cross-cycle semantic dedup (embedding cache)	§5.3, §6.4
Security hardening checklist	§17.6
Freshness expiry + slot replacement for queued posts	§12.4, §17.2

18. Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
Free-tier LLM quota exhausted mid-cycle	Medium	Medium	Batch gate ≤ 15 articles/run; split across 2 days; Groq fallback for short tasks; Ollama local fallback; auto-resume on 429 with backoff.
Instagram app review delays / rejection	High (delays)	Medium	Start week 1; use development-mode publishing to your own account meanwhile; Business Suite manual posting as fallback.
Meta token expiry	Certain (60-day)	High	Watchtower token-age alert at 7 days; monthly re-exchange calendar task; long-lived token helper workflow.
Fact-check misses $\rightarrow$ wrong info published	Low	High	Dual-source corroboration rule; auto-flag topics; quarantine > 2 rewrites; sources footer on every post; 24 h human digest during test phase.
RSS feed changes / dead feeds	Medium	Low	Continue-on-fail; coverage guard; monthly feed audit checklist item.
Image API downtime (Pollinations)	Medium	Low	Retry 2x; deterministic Pillow fallback art; deck never missing cards.
Oracle VM unavailable (capacity)	Medium	Medium	Docker compose = portable; fallback home PC / Raspberry Pi; Cloudflare Tunnel works from any host; SQLite file backups.
Algorithm/engagement stagnation	Medium	Medium	KPI dashboard (Section 19); A/B captions; daypart auto-tuning; topic mix caps keep feed varied.
AI slop perception / follower backlash	Medium	Medium	Disclose AI-assisted + “verified” footer; editorial quality gates; human oversight during Phase 1.
Stale-news backlog (pool outgrows 3 posts/day)	Certain	Medium	Queue Manager: priority $\times$ freshness re-ranking, 72 h expiry, pool cap with graceful aging-out to audit.
Instagram rejects image_url (private/expired tunnel URL)	Medium	High	Cards served from Cloudflare R2 public bucket (stable HTTPS, zero egress fees); tunnel never used for media.
Both cloud LLM providers down	Low	Medium	Ollama local fallback on the 24 GB VM covers every text task; Watchtower provider health flag auto-switches.
Rubric / fact-checker drift or bias	Medium	Medium	Versioned rubric + golden-set regression tests after every prompt change; score distribution monitored weekly.
n8n bug or node drift after update	Low	Medium	Pin Docker image version; export workflow JSON backups weekly (n8n native); test after upgrades.

19. 30-Day Build & Test Plan

Day	Tasks — each ends with a demo-able result
1-2	Provision Oracle Always Free ARM VM (or home PC); install Docker; deploy n8n Community Edition via compose; secure with tunnel URL + password; install Python deps (Pillow, sqlite3 built-in) and create <code>/data</code> , <code>/media</code> , <code>/audit</code> , <code>/backups</code> .
3	Get Gemini API key + Groq key; test 1 Gemini call and 1 Groq call from n8n; verify free-tier numbers against dashboard.
4-5	Build <code>0W-01 scrape</code> : 16 RSS branches, normalize, hash, SQLite insert. Demo: run → ≥100 rows in <code>headlines</code> .
6-7	Create SQLite schema script (Section 13); run migrations; write <code>init_db.py</code> ; seed <code>config</code> (theme, thresholds, slots).
8-9	Build <code>0W-02 score</code> : rubric prompt, batch loop, weighted scoring, topic caps, semantic dedup test on 50 headlines. Demo: 30-40 scored rows with subscores.
10-12	Build <code>0W-03 write</code> : research call, 8-section loop, assembly, word-count gate. Demo: 3 sample articles ≥ 3,000 words; manual read-through.
13-15	Build <code>0W-04 factcheck</code> : claim extraction, Google News RSS search loop, verdict, REWRITE loop. Demo: 3 articles pass, 1 forced rewrite works end-to-end.
16	Build <code>0W-05 group</code> : audit IDs, bundle JSON, <code>/audit</code> export.
17-19	Design themes in Canva (free) → encode <code>theme.json</code> ; build <code>render_cards.py</code> ; test Pollinations art + fallback. Demo: 10-card deck for 1 group, on-theme.
20	Build <code>0W-07 caption</code> with length gate; 2 variants per post. Build <code>0W-12 insights</code> pull and verify one Insights call.
21-23	Meta setup: Business/Creator IG, Facebook Page link, Developer app, permissions, app review submission (started day 1 ideally!), long-lived token helper.
24-25	Build <code>0W-08 schedule</code> : slot engine, container creation, publish, permalink, guards. Demo (dev mode): publish 1 carousel to your own IG account via API.
26-27	Build <code>0W-09 watchtower</code> + <code>0W-11 errors</code> + Telegram bot commands; wire daily digest. Build <code>0W-13 review</code> bridge and <code>0W-14 queue</code> manager; run rubric + fact-checker golden-set tests.
28-29	Full dry run: 2 complete cycles with auto-publish OFF; human review of every deck & caption via OW-13; phone test of 3 decks (zoom-readability); fix gates; measure timings.
30	Go-live: auto-publish ON for 3 posts/day; launch checklist (Appendix C) sign-off; start KPI tracking.

20. Channel KPIs & Targets

KPI	Target (90 days)	Measured via
Posts published / week	18-21 (3/day)	<code>posts</code> table
Carousel completion rate	> 35%	IG Insights
Reach / post	grow 15%/wk	IG Insights
Engagement rate (likes+comments+shares)/reach	> 5%	IG Insights
Follower growth	> 200/mo	IG Insights
Fact-check pass rate	> 85% first pass	<code>fact_checks</code>
Rewrite rate	< 30%	<code>articles.draft_rev</code>
Quarantine rate	< 5%	<code>fact_checks</code>
Publish success rate	> 99%	<code>posts</code>
Pipeline uptime (no cycle missed)	> 97%	Watchtower
Cost	\$0.00/mo	Strictly no paid services in stack
Median headline age at publish	< 30 h	<code>posts</code> joined with <code>headlines</code>

21. Legal & Compliance

- **Copyright:** news facts are not copyrightable, but wording and images are. The writer always paraphrases and re-reports; direct quotes are short, attributed, and sourced. Never reproduce full articles or outlet photos. AI-generated art (Pollinations) is used as background texture only.
- **Attribution & transparency:** every post carries a sources footer and a standard “AI-assisted & fact-checked” line. Many regions/platforms now require disclosure of AI-generated content — Meta asks users to label AI content; comply.
- **Defamation & accuracy:** the fact-checker’s dual-source rule, auto-flag topics, quarantine, and the 24 h human digest during the test phase are the legal backstop. Retraction procedure: any user-flagged error → Watchtower alert → correct or delete within 24 h.
- **Platform ToS:** Instagram prohibits spam (volume, repetition, engagement-bait). OverWatch: 3 posts/day, no engagement-bait phrasing (“like if”), no hashtag stuffing, human-verifiable content. Keep automation within API limits (25 posts/24 h).
- **RSS ToS:** most feeds permit programmatic reading for non-commercial redistribution with attribution; Google News RSS is for personal/non-commercial use — this channel qualifies as non-commercial at launch; re-check if the channel becomes commercially monetized, and swap to publisher APIs or licensed feeds if needed.
- **Data protection:** no personal data is collected about followers; analytics are aggregate only.

Appendix A — n8n Node Cheat-Sheet

n8n node	Used for	Tips
Schedule Trigger	All agent wake-ups	Use cron expressions; keep timezone = server UTC, convert to audience time in code.
RSS Read	Feed ingestion	Per-feed branch; continue-on-fail; 20 s timeout.
Code (Python/JS)	All SQLite, normalization, gating, rendering calls	Python for Pillow; JS for light transforms; reuse <code>db.py</code> helper via Code node import.
AI Agent	LLM tasks (scoring, writing, verdicts, captions)	Structured output = JSON schema in system prompt; temperature 0 for scores/verdicts, 0.7 for writing.
HTTP Request	Groq, Pollinations, Graph API, source fetches, Google News search	Set sane timeouts (image gen 60 s); retry-on-fail with backoff for 429/5xx.
Split In Batches	Chunking 20 headlines / 15 articles per run	Paired with the quota pacing rule.
Loop Over Items	Per-article sections, per-claim verification	Use “loop continue” flags; cap iterations (claims $\leq$ 12).
Execute Command	Pillow render script, sqlite backup, ffmpeg (Phase 2)	Point at VM scripts; capture stdout/stderr into <code>audit_log</code> .
Telegram	Alerts, digest, commands	Trigger via webhook + token validation.
Error Trigger	Global error alerting	Dedupe by workflow+time window; alert, never silently swallow.
Wait	Pacing between LLM batches	Useful to flatten request bursts under RPM limits.

Appendix B — Prompt Library (copy-paste)

B.1 Scoring rubric prompt (temperature 0)

You are the SCORING AGENT of an automated news channel. Score each headline using EXACTLY this rubric. Output JSON only:

```
{
  "scores": [
    {"headline_id": "...", "curiosity": 0-10, "emotion": 0-10,
     "relevance": 0-10, "freshness": 0-10, "visual": 0-10,
     "authority": 0-10, "shareability": 0-10, "rationale": "<1 sentence>"}
  ]
}
```

Rubric definitions:

1. curiosity (20%): information gap that demands a click. 10 = question or contrast; 0 = flat statement.
2. emotion (15%): surprise, awe, inspiration, FOMO. Outrage bait scores DOWN. 10 = universally felt; 0 = dry bulletin.
3. relevance (15%): direct effect on reader's money, work, health, life.
4. freshness (15%): <6h = 10; 6-24h = 7; 24-48h = 3.
5. visual (15%): can this story be told in 10 beautiful cards? maps, photos, charts > abstract policy.
6. authority (10%): credible outlet; likely 2+ independent sources exist.
7. shareability (10%): would readers forward it? identity-relevant, conversational, must-know framing.

Rules: no speculation as fact; mark [UNVERIFIED] if the headline's claim cannot be attributed; never invent numbers. Return valid JSON, nothing else.

B.2 Writer outline prompt (temperature 0.3)

You are the RESEARCH lead of a news channel. Given the headline and the source article below, produce a JSON outline:

```
{
  "angle": "one-sentence editorial angle",
  "standfirst": "1-sentence promise to the reader",
  "sections": ["8 section titles in narrative order"],
  "key_facts": ["5-10 verifiable facts with numbers/dates/names"],
  "quotes_to_verify": ["short quotable statements found in the source"],
  "open_questions": ["2-3 questions the article can raise"]
}
```

Constraints: 8 sections; each section 400-500 words; facts must appear in the source article or be marked UNVERIFIED; no invented quotes.

Headline: {{headline}}

Source text: {{source_text}}

B.3 Writer section prompt (temperature 0.7)

You are a senior journalist writing for a global audience. Write SECTION {{n}} of the article: "{{section_title}}".

Outline: {{outline}}

Previously written sections: {{prior_sections}}

Style rules:

- 400-500 words; grade-8 reading level; short sentences.
- Numbers over adjectives; named people; active voice.
- One idea per paragraph; a bolded takeaway every 3-5 paragraphs.
- No invented facts, quotes, or stats. Mark uncertainty explicitly.
- End with a bridge sentence into the next section.

Output only the section text, no preamble.

B.4 Fact-check claim extraction (temperature 0)

Extract every verifiable factual claim from this article (max 12):
numbers, dates, names, attributions, causal statements.
Output JSON:
[{"claim": "...", "type": "statistic|event|attribution|causal|identity",
"importance": "high|medium|low"}]
Skip opinions, predictions clearly labeled, and generic statements.

B.5 Fact-check verdict (temperature 0)

You are a strict fact-checker. For each claim, given the evidence excerpts gathered, output JSON:
[{"claim": "...", "verdict": "supported|unsupported|contradicted|unverifiable",
"confidence": 0.0-1.0, "evidence": "1-line note with source names"}]
Rules: two independent high-trust sources = corroborated; anonymous blogs and aggregators are NOT evidence; if evidence conflicts, mark "contradicted"; if nothing relevant found, "unverifiable".

B.6 Caption generator (temperature 0.7)

Write an Instagram caption for this carousel post.
Input: headline, standfirst, 5 key facts, card-1 text, theme, issue number.
Output JSON:
{
"hook": "1-2 lines, surprising fact or question, never 'Breaking:',"
"summary": "3-5 lines covering who/what/why it matters",
"value_line": "line telling readers to swipe for the full story",
"cta": "exactly one: question | save | follow (rotate by input)",
"hashtags": "3-8 tags: 1 broad, 1 topic, 1 niche, 1 branded",
"sources_line": "'Sources: ... – verified.'",
"full_caption": "hook + summary + value_line + cta + hashtags + sources_line"}
Constraints: full_caption ≤ 2000 chars; no emoji spam (max 3); no engagement-bait phrases like 'like if you agree'.

Appendix C — Kickoff Checklist (printable)

☐	Item
☐	Instagram switched to Business/Creator account (free, in-app) — start immediately, it takes time.
☐	Facebook Page created + linked to Instagram account.
☐	Meta Developer app created; <code>instagram_basic</code> + <code>instagram_content_publish</code> requested; app review submitted.
☐	Long-lived token obtained (60 days) + token-refresh reminder in Watchtower.
☐	Oracle Always Free ARM VM provisioned (or fallback home PC).
☐	Docker + n8n Community Edition running; tunnel URL working; admin password set.
☐	Python env: Pillow installed; <code>/data /media /audit /backups</code> created; <code>init_db.py</code> run.
☐	Gemini API key (<code>aistudio.google.com/apikey</code>) + Groq key (<code>console.groq.com</code>) tested in n8n.
☐	Telegram bot created (BotFather); token + chat ID in n8n; <code>/status</code> responds.
☐	Theme approved (NewsBlue default) and encoded in <code>theme.json</code> .
☐	All 11 workflows imported from JSON backups; cron schedules set (UTC).
☐	Dry-run cycle completed with auto-publish OFF; decks + captions human-reviewed.
☐	First live carousel published via API (dev mode, own account).
☐	Daily digest reviewed for 3 consecutive days before full auto mode.
☐	KPI baseline recorded (Section 19) — go-live date stamped.

Final word. OverWatch is deliberately boring in engineering and relentless in operations: fixed rubrics, deterministic gates, one database, one theme, alerts for everything. That boring core is what lets a \$0 stack run a credible news channel 24/7 — and every upgrade path in Phase 2 keeps the same architecture. Build it once, watch it run.